

C PROGRAMMING

Complete Learning Notes

From Beginner to Advanced Mastery

Based on Purbanchal University & Pokhara University Curriculum

BCE 1001 | BIT175CO | BCA202603

Prepared by an Expert Tutor | 15+ Years of Teaching Experience

■ Contents covered: Problem Solving · C Fundamentals · Operators · I/O · Control Statements · Functions · Arrays · Pointers · Structures · Files · Graphics

TABLE OF CONTENTS

#	Topic	Stage
1	Learning Roadmap Overview	Foundation
2	What is Programming? (Analogies & Real-World)	Stage 0
3	Problem Solving & Algorithms	Stage 1
4	Introduction to C Language	Stage 2
5	C Fundamentals (Data Types, Variables)	Stage 2
6	Operators & Expressions	Stage 3
7	Input & Output	Stage 3
8	Control Statements (Loops, Conditions)	Stage 4
9	Functions	Stage 5
10	Arrays & Strings	Stage 6
11	Pointers	Stage 7
12	Structures & Unions	Stage 8
13	File Handling	Stage 9
14	Dynamic Memory Allocation	Stage 9
15	12-Week Structured Curriculum	All Stages
16	20+ Mini-Project Ideas	Practice
17	Past Exam Question Analysis	Exam Prep

SECTION 1: THE COMPLETE LEARNING ROADMAP

This roadmap is designed based on the official Purbanchal University syllabus (BCE 1001 / BIT175CO) and Pokhara University curriculum, covering all 12 chapters over 45 lecture hours. Think of learning programming like building a house — you must lay the foundation before raising walls.

Stage	Topic	Duration	Focus
■■ Stage 0	Mindset & Setup	1 week	Install compiler, understand what code is, set up your workspace
■ Stage 1	Problem Solving	1 week	Algorithms, flowcharts, pseudocode — think before you code
■ Stage 2	C Basics	2 weeks	Structure, data types, variables, constants, keywords
■ Stage 3	Operators & I/O	1 week	All operators, printf/scanf, format specifiers
■ Stage 4	Control Flow	2 weeks	if-else, switch, for/while/do-while, break, continue
■■ Stage 5	Functions	2 weeks	User-defined, recursion, scope, storage classes
■ Stage 6	Arrays & Strings	2 weeks	1D/2D arrays, string functions, sorting
■ Stage 7	Pointers	2 weeks	Pointer arithmetic, arrays+pointers, DMA
■■ Stage 8	Structures & Unions	1 week	struct, union, nested structs, typedef
■ Stage 9	Files & Advanced	1 week	File I/O, binary files, error handling
■ Stage 10	Projects & Mastery	Ongoing	Build real projects, attempt past papers, optimize code

SECTION 2: WHAT IS PROGRAMMING?

2.1 The Cooking Analogy

Imagine you are cooking dal bhat. You follow a recipe: first add water, then rice, then heat for 20 minutes. If the dal is too thick, add more water. This recipe IS a program. The computer is your kitchen, and C code is the recipe written in a language the kitchen understands.

2.2 What is a Computer Program?

A computer program is a set of step-by-step instructions given to a computer to perform a specific task. Just as you give directions to someone ('go straight, turn left at the temple, stop at the blue house'), you give instructions to a computer in a programming language.

2.3 Why Learn C?

Reason	Analogy
C is the foundation of most languages	Like learning Sanskrit helps you understand Nepali, Hindi & Bengali
C is fast and efficient	Like driving a manual car — full control, maximum speed
C is used everywhere	Operating systems (Windows/Linux), embedded systems, games
University & exam requirement	Core subject in BE Computer/Electronics/BCA/BIT programs
Career advantage	Systems programming, firmware, competitive coding

2.4 How Does C Code Become a Running Program?

The journey of your C code has four stages:

Step	Stage	What Happens	Tool
1	Source Code (.c)	You write instructions in C language	Text editor / IDE
2	Compilation	Compiler checks for errors, translates to machine code	GCC / Turbo C
3	Linking	Connects libraries (like stdio.h) to your program	Linker
4	Execution	CPU runs your .exe file, produces output	Operating System

■ **Note:** Think of it like a Nepali-to-English translator: you write in Nepali (C code), the compiler translates to English (machine code), and the computer reads English.

SECTION 3: PROBLEM SOLVING, ALGORITHMS & FLOWCHARTS

3.1 Problem Analysis (Chapter 1 of Syllabus)

Before writing a single line of code, a good programmer THINKS. Problem analysis means understanding: What is the input? What is the expected output? What are the constraints?

Real-World Example: Finding the largest of 3 numbers

- Input: Three numbers (say: 45, 72, 33)
- Process: Compare them to find the maximum
- Output: Print '72 is the largest'

3.2 Algorithm

An algorithm is a finite set of well-defined instructions to solve a problem. Rules: (1) Unambiguous steps, (2) Has a definite start and end, (3) Produces a result, (4) Feasible.

Algorithm: Find Largest of 3 Numbers

- Step 1: START
- Step 2: Read three numbers A, B, C
- Step 3: IF $A > B$ AND $A > C$ THEN $large = A$
- Step 4: ELSE IF $B > C$ THEN $large = B$
- Step 5: ELSE $large = C$
- Step 6: PRINT $large$
- Step 7: STOP

3.3 Flowchart Symbols

Symbol	Shape	Purpose
Oval	START / STOP	Marks the beginning or end of a program
Rectangle	PROCESS	Arithmetic operations, assignments
Diamond	DECISION	Yes/No questions, conditions (if-else)
Parallelogram	INPUT/OUTPUT	Read from user or print to screen
Arrow	FLOW LINE	Shows direction of program flow
Circle	CONNECTOR	Connects parts of a large flowchart

3.4 Compilation & Execution Process

Preprocessing → Compilation → Assembly → Linking → Loading → Execution

■ **Note:** Bugs are errors in your program. Syntax errors = spelling mistakes in code. Logic errors = wrong recipe. Runtime errors = program crashes while running.

SECTION 4: INTRODUCTION TO C LANGUAGE

4.1 Brief History

C was developed by Dennis Ritchie at Bell Labs (1972). It evolved from B language (Ken Thompson). C was used to write the UNIX operating system. It is called a 'middle-level language' because it has features of both high-level languages (easy to write) and low-level languages (direct hardware access).

4.2 Basic Structure of a C Program

```
/* ■■ Basic C Program Structure ■■ */
#include <stdio.h> // Preprocessor directive
#include <conio.h> // For getch(), clrscr()

int main() // Main function - entry point
{
    // Variable declarations
    int num = 10;

    // Statements
    printf("Hello, World!\n");
    printf("Number = %d\n", num);

    getch(); // Wait for key press
    return 0; // Return to OS
}
```

Parts of a C Program:

Part	Description	Example
Documentation	Comments for human readers	/* This finds sum */
Preprocessor	Commands before compilation	#include
Global Declarations	Variables accessible everywhere	int count = 0;
main() function	Entry point of every C program	int main() { }
Subprograms	User-defined functions	int add(int a, int b)

SECTION 5: C FUNDAMENTALS

5.1 Character Set

C uses the following characters:

- Letters: A-Z, a-z
- Digits: 0-9
- Special Characters: + - * / % = () { } [] ; , . # & | ! ~ ^ < >
- White Spaces: space, tab (\t), newline (\n), carriage return

5.2 Keywords (32 Reserved Words)

Keyword 1	Keyword 2	Keyword 3	Keyword 4	Keyword 5	Keyword 6	Keyword 7	Keyword 8
auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

■ **Note:** Keywords CANNOT be used as variable names. They have fixed meanings in C.

5.3 Data Types

Data Type	Size	Range	Format Specifier	Example
int	2 or 4 bytes	-32768 to 32767	%d	int age = 20;
long int	4 bytes	-2,147,483,648 to +2,147,483,647	%ld	long pop = 28000000;
float	4 bytes	3.4e-38 to 3.4e+38	%f	float pi = 3.14;
double	8 bytes	1.7e-308 to 1.7e+308	%lf	double e = 2.718;
char	1 byte	-128 to 127	%c	char grade = 'A';
unsigned int	2 or 4 bytes	0 to 65535	%u	unsigned int x=50;
void	0 bytes	No value	N/A	void display();

5.4 Variables & Constants

A variable is a named memory location that can hold different values during program execution. Think of it like a labeled box — the label is the variable name, the box contents can change.

```
// Variable declaration and initialization
int age = 20;
float salary = 45000.50;
char grade = 'A';
double pi = 3.14159265;
```



```
// Constants (value cannot change)
const float TAX_RATE = 0.13; // const keyword
#define MAX_SIZE 100 // symbolic constant
#define PI 3.14159
```

5.5 Rules for Variable Naming

- Must begin with a letter (a-z, A-Z) or underscore (_)
- Can contain letters, digits, and underscores — no spaces or special chars
- Cannot be a keyword
- Case-sensitive: 'age', 'Age', and 'AGE' are three different variables
- Max length: typically 31 characters (ANSI C)

Valid Names ■	Invalid Names ■	Reason for Invalidity
age	2age	Cannot start with digit
_total	my age	Contains space
student_name	int	Is a keyword
rollNo	roll#no	# is not allowed
MAX_SCORE	my-var	Hyphen not allowed

5.6 Type Conversion

Implicit (automatic): The compiler converts smaller type to larger type automatically. Explicit (casting): The programmer forces conversion.

```
// Implicit conversion
int a = 5; float b = 2.5;
float result = a + b; // a is auto-converted to float = 7.5

// Explicit casting
int x = 10, y = 3;
float ans = (float)x / y; // Cast x to float → 3.333
```

SECTION 6: OPERATORS & EXPRESSIONS

6.1 Types of Operators

Category	Operators	Example	Result
Arithmetic	+ - * / %	10 % 3	1 (remainder)
Relational	== != > < >= <=	5 > 3	1 (true)
Logical	&& !	5>3 && 2<4	1 (true)
Assignment	= += -= *= /= %=	x += 5	x = x + 5
Increment/Decrement	++ --	x++ or ++x	adds 1 to x
Bitwise	& ^ ~ << >>	5 & 3	1
Conditional (Ternary)	? :	a>b ? a : b	max value
sizeof()	sizeof()	sizeof(int)	2 or 4
Address/Pointer	& *	&x;	address of x
Comma	,	x=1, y=2	evaluates both

6.2 Prefix vs Postfix (Very Important!)

```
int a = 5, b, c;

b = ++a; // PREFIX: a becomes 6 FIRST, then b = 6
// result: a=6, b=6

c = a++; // POSTFIX: c gets current a=6 FIRST, then a becomes 7
// result: a=7, c=6

// Classic exam question:
int a=5, b,c;
b = ++a + a++; // b = 6 + 6 = 12, a = 7 (compiler-dependent)
```

■ **Note:** Prefix (++a): Increment FIRST, use later. Postfix (a++): Use FIRST, increment later. Remember: 'Pre = prepare first'.

6.3 Operator Precedence (High to Low)

Level	Operators	Associativity
1 (Highest)	() [] -> .	Left to Right
2	! ~ ++ -- + - * & (unary) sizeof	Right to Left
3	* / %	Left to Right
4	+ -	Left to Right
5	<< >>	Left to Right

Level	Operators	Associativity
6	< <= > >=	Left to Right
7	== !=	Left to Right
8	&	Left to Right
9	^	Left to Right
10		Left to Right
11	&&	Left to Right
12		Left to Right
13	?:	Right to Left
14 (Lowest)	= += -= *= /=	Right to Left

SECTION 7: INPUT & OUTPUT (I/O)

7.1 Format Specifiers

Specifier	Data Type	Example
%d	int	printf("%d", 42);
%ld	long int	printf("%ld", 100000L);
%f	float (6 decimal places)	printf("%f", 3.14);
%lf	double	printf("%lf", 3.14159);
%.2f	float with 2 decimal places	printf("%.2f", 3.14159); → 3.14
%c	char (single character)	printf("%c", 'A');
%s	string (char array)	printf("%s", "Nepal");
%x	hexadecimal	printf("%x", 255); → ff
%o	octal	printf("%o", 8); → 10
%e	scientific notation	printf("%e", 12345.0); → 1.234500e+04
%20d	right-align in 20-char field	42 (padded with spaces)
%-20d	left-align in 20-char field	42 (padded right)
%05d	zero-padded	00042

7.2 Formatted I/O

```
// printf – formatted output
printf("Name: %s, Age: %d, GPA: %.2f\n", name, age, gpa);

// scanf – formatted input (use & for non-string)
int age; float salary; char name[50];
scanf("%d", &age);
scanf("%f", &salary);
scanf("%s", name); // NO & for arrays
scanf("%d %f", &age, &salary); // multiple inputs
```

7.3 Unformatted I/O

Function	Purpose	Syntax	Note
getchar()	Read one character	ch = getchar();	Waits for Enter
putchar()	Print one character	putchar(ch);	Prints single char
gets()	Read a string with spaces	gets(str);	Unsafe — deprecated

Function	Purpose	Syntax	Note
puts()	Print a string + newline	puts(str);	Adds newline auto
getch()	Read char without echo/Enter	ch = getch();	Turbo C only
getche()	Read char with echo, no Enter	ch = getche();	Turbo C only
putch()	Print char (no newline)	putch(ch);	Turbo C only

7.4 Escape Sequences

Sequence	Meaning	Use
\n	Newline	Move to next line
\t	Horizontal Tab	Indent text
\\	Backslash	Print \ character
\"	Double Quote	Print " in string
\0	Null Character	String terminator
\a	Alert/Bell	Sound a beep
\b	Backspace	Move cursor back
\r	Carriage Return	Go to start of line

SECTION 8: CONTROL STATEMENTS

8.1 Conditional Statements

```
// 1. Simple if
if (marks >= 40)
printf("Pass\n");

// 2. if-else
if (marks >= 40)
printf("Pass\n");
else
printf("Fail\n");

// 3. if-else if ladder
if (marks >= 90) printf("A+");
else if (marks >= 80) printf("A");
else if (marks >= 70) printf("B");
else if (marks >= 60) printf("C");
else if (marks >= 40) printf("D");
else printf("Fail");

// 4. Nested if
if (age >= 18) {
if (hasID == 1)
printf("Can vote\n");
}

// 5. Ternary operator (compact if-else)
int max = (a > b) ? a : b;
```

8.2 switch-case

```
switch (choice) {
case 1: printf("Addition"); break;
case 2: printf("Subtraction"); break;
case 3: printf("Multiplication"); break;
default: printf("Invalid choice");
}
// IMPORTANT: Always use 'break' or execution falls through!
```

8.3 Loops

Loop	Type	When to Use	Syntax
for	Entry-controlled	Count known in advance	for(init; cond; update) { }
while	Entry-controlled	Count unknown, check first	while(condition) { }
do-while	Exit-controlled	Must execute at least once	do { } while(condition);

```
// for loop: print 1 to 10
for (int i = 1; i <= 10; i++)
printf("%d ", i);

// while loop: sum until 0 entered
int n, sum = 0;
scanf("%d", &n);
while (n != 0) {
sum += n;
scanf("%d", &n);
}

// do-while: menu that runs at least once
do {
printf("1.Add 2.Sub 3.Exit: ");
scanf("%d", &choice);
} while (choice != 3);
```

8.4 Jump Statements

Statement	Purpose	Where Used
break	Exit loop or switch immediately	Inside for/while/do-while/switch
continue	Skip current iteration, continue loop	Inside for/while/do-while
goto label	Jump to any labeled statement	Anywhere (use sparingly!)
exit(0)	Terminate entire program	Any function (needs stdlib.h)
return	Return value and exit function	Inside functions

■ **Note:** Avoid goto in modern programming. It makes code hard to follow (called 'spaghetti code'). Prefer structured loops instead.

SECTION 9: FUNCTIONS

9.1 Why Functions?

Think of functions like departments in a company. The CEO (main) doesn't do all the work — they delegate to HR, Finance, and Operations. Each department does its job and reports back. Functions make code: Reusable, Readable, Debuggable, and Maintainable.

9.2 Function Anatomy

```
// Function PROTOTYPE (declaration) – placed before main()
int add(int a, int b);

// MAIN function
int main() {
    int result = add(10, 20); // Function CALL
    printf("%d", result); // Output: 30
    return 0;
}

// Function DEFINITION
int add(int a, int b) { // return_type name(parameters)
    return a + b; // return statement
}
```

9.3 Call by Value vs Call by Reference

Feature	Call by Value	Call by Reference
What is passed	Copy of the value	Address of the variable
Original changed?	No — original is safe	Yes — original is modified
Use case	When you don't want to modify original	When you need to modify original
Example	swap_wrong(a,b)	swap_correct(&a,&b;)
Syntax	void f(int x)	void f(int *x)

```
// Call by Value – won't swap!
void swap_wrong(int a, int b) {
    int temp = a; a = b; b = temp; // only local copies swapped
}

// Call by Reference – works!
void swap(int *a, int *b) {
    int temp = *a; *a = *b; *b = temp;
}

// Call: swap(&x;, &y;);
```


9.4 Recursive Functions

Recursion: A function that calls itself. Like Russian nesting dolls — each doll contains a smaller version of itself. Every recursion MUST have: (1) Base case (stopping condition), (2) Recursive case.

```
// Factorial using recursion
long factorial(int n) {
if (n == 0 || n == 1) // BASE CASE
return 1;
return n * factorial(n-1); // RECURSIVE CASE
}
// factorial(5) = 5 * 4 * 3 * 2 * 1 = 120

// Fibonacci using recursion
int fib(int n) {
if (n <= 1) return n; // base: fib(0)=0, fib(1)=1
return fib(n-1) + fib(n-2);
}
```

9.5 Variable Scope & Storage Classes

Class	Declared	Lifetime	Default Value	Keyword
Auto (local)	Inside function/block	Until block ends	Garbage	auto (default)
Global	Outside all functions	Entire program	0	(no keyword needed)
Static	Inside function	Entire program run	0	static
Register	Inside function	Until block ends	Garbage	register
External	Another file	Entire program	0	extern

SECTION 10: ARRAYS & STRINGS

10.1 What is an Array?

An array is a collection of variables of the same data type stored in contiguous memory locations. Analogy: A row of lockers in a school — all numbered, all same size, all in a row.

```
// 1D Array declaration and initialization
int marks[5]; // declare (uninitialized)
int marks[5] = {85, 72, 90, 68, 78}; // declare + initialize
int marks[] = {85, 72, 90, 68, 78}; // size auto-determined

// Accessing elements (index starts at 0!)
printf("%d", marks[0]); // 85 (first)
printf("%d", marks[4]); // 78 (last)

// Traversal with loop
for (int i = 0; i < 5; i++)
    printf("%d ", marks[i]);

// 2D Array (matrix) - 3 rows, 4 columns
int matrix[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
printf("%d", matrix[1][2]); // Row 1, Col 2 → 7
```

10.2 Passing Arrays to Functions

```
// Arrays are passed by reference automatically!
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
}
// Call: printArray(marks, 5);
```

10.3 Strings in C

A string is an array of characters terminated by a null character '\0'. There is no dedicated string type in C — we use char arrays.

```
char name[20] = "Nepal";
// Stored as: N e p a l \0 (null terminator is automatic)

char name[20]; scanf("%s", name); // Read string (stops at space)
gets(name); // Read full line (with spaces)
```

10.4 String Handling Functions (string.h)

Function	Purpose	Example
strlen(s)	Length of string	strlen("Nepal") → 5

Function	Purpose	Example
strcpy(dst,src)	Copy src to dst	strcpy(b, a); // b gets value of a
strcat(dst,src)	Append src to end of dst	strcat(s1, " World")
strcmp(s1,s2)	Compare (0 if equal, >0 if s1>s2)	if(strcmp(a,b)==0)
strrev(s)	Reverse a string	strrev("Nepal") → lapeN
strupr(s)	Convert to UPPERCASE	strupr("nepal") → NEPAL
strlwr(s)	Convert to lowercase	strlwr("NEPAL") → nepal
strstr(s,sub)	Find substring	strstr("Hello World","World")

SECTION 11: POINTERS

11.1 What is a Pointer?

A pointer is a variable that stores the memory ADDRESS of another variable. Analogy: Think of RAM as a huge apartment building. Each room has an address (like 1024, 1025...). A normal variable IS the person living in the room. A pointer is a piece of paper with the ROOM NUMBER written on it.

```
int x = 10; // x stores the value 10 at some address
int *ptr = &x; // ptr stores the ADDRESS of x

printf("%d", x); // 10 (value of x)
printf("%p", &x); // address of x (e.g. 0x7fff5c4)
printf("%p", ptr); // same address as &x;
printf("%d", *ptr); // 10 (value AT the address = dereferencing)

*ptr = 25; // Change x through pointer
printf("%d", x); // Now x = 25 !
```

11.2 Pointer Concepts

Concept	Description	Example
& operator	Address-of — gets memory address	&x; → address of x
* operator	Dereference — gets value at address	*ptr → value at ptr
NULL pointer	Pointer pointing nowhere (safe default)	int *p = NULL;
Wild pointer	Uninitialized pointer — DANGEROUS!	int *p; // unknown address
Void pointer	Generic pointer — no specific type	void *p;
Pointer to pointer	Pointer storing address of another pointer	int **pp = &ptr;

11.3 Pointers and Arrays

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // p points to arr[0]

printf("%d", *p); // 10 (arr[0])
printf("%d", *(p+1)); // 20 (arr[1])
printf("%d", *(p+2)); // 30 (arr[2])

// arr[i] is SAME as *(arr + i)
// Pointer arithmetic moves by sizeof(data type)
```

11.4 Dynamic Memory Allocation (DMA)

DMA lets you allocate memory at runtime — you don't need to know the size at compile time. Like reserving a hotel room on the spot rather than booking in advance.

Function	Purpose	Syntax	Header
malloc(n)	Allocate n bytes, uninitialized	int *p = (int*)malloc(n*sizeof(int));	stdlib.h
calloc(n,s)	Allocate n blocks of s bytes, initialized to 0	int *p = (int*)calloc(n,sizeof(int));	stdlib.h
realloc(p,n)	Resize previously allocated block	p = (int*)realloc(p, new_size);	stdlib.h
free(p)	Release allocated memory	free(p); p = NULL;	stdlib.h

```
int n, *arr;
printf("Enter size: "); scanf("%d", &n);
arr = (int*) malloc(n * sizeof(int));
if (arr == NULL) { printf("Memory failed!"); exit(1); }
// use arr as normal array...
free(arr); // ALWAYS free when done!
```

SECTION 12: STRUCTURES & UNIONS

12.1 Structure

A structure is a user-defined data type that groups variables of different types under one name. Analogy: A student record form — it has Name (string), Roll No (int), Marks (float) all together.

```
// Define structure
struct Student {
int rollNo;
char name[50];
float marks;
};

// Declare and use
struct Student s1;
strcpy(s1.name, "Ram Sharma");
s1.rollNo = 101;
s1.marks = 87.5;
printf("%s got %.1f marks\n", s1.name, s1.marks);

// Array of structures
struct Student class[60];
scanf("%s", class[0].name);

// Pointer to structure
struct Student *ptr = &s1;
printf("%s", ptr->name); // Use -> for pointer access
```

12.2 typedef with struct

```
// typedef makes code cleaner
typedef struct {
int day, month, year;
} Date;

Date dob = {15, 8, 2000}; // No 'struct' keyword needed
printf("%d/%d/%d", dob.day, dob.month, dob.year);
```

12.3 Structure vs Union

Feature	Structure	Union
Memory	Each member has its OWN memory	ALL members SHARE the same memory
Size	Sum of all member sizes	Size of the LARGEST member only
Simultaneous access	All members can be used at once	Only ONE member at a time
Keyword	struct	union

Feature	Structure	Union
Use case	Store complete records	Memory-saving when only one field needed at a time
Example	Student record (all fields)	Data type variant (int OR float OR char)

```

union Data {
int i; // 4 bytes
float f; // 4 bytes
char c; // 1 byte
}; // Size = 4 (largest member)
// Only ONE can be used at a time!

```

SECTION 13: FILE HANDLING

13.1 File Concept

Files allow permanent storage of data. Variables lose data when program ends; files don't. Think of variables as a whiteboard (temporary) and files as a notebook (permanent).

13.2 File Opening Modes

Mode	Meaning	File Exists?	File Missing?
"r"	Read text file	Opens for reading	Error (returns NULL)
"w"	Write text file	Overwrites existing	Creates new file
"a"	Append text file	Adds to end	Creates new file
"r+"	Read+Write	Opens both modes	Error
"w+"	Write+Read (overwrite)	Overwrites	Creates new
"a+"	Append+Read	Adds to end	Creates new
"rb"	Read binary file	Opens binary	Error
"wb"	Write binary file	Overwrites binary	Creates new

13.3 File Functions

```
FILE *fp;

// Open & check
fp = fopen("data.txt", "w");
if (fp == NULL) { printf("Cannot open file!"); exit(1); }

// Write text
fprintf(fp, "Name: %s, Age: %d\n", name, age);
fputs("Hello File!\n", fp);
fputc('A', fp);

// Close (ALWAYS close!)
fclose(fp);

// Read text
fp = fopen("data.txt", "r");
char line[100];
while (fgets(line, 100, fp) != NULL)
    printf("%s", line);
fclose(fp);

// Random access
fseek(fp, 0, SEEK_SET); // go to beginning
```



```
fseek(fp, 0, SEEK_END); // go to end
ftell(fp); // current position
rewind(fp); // same as fseek(fp,0,SEEK_SET)

// Error checking
if (feof(fp)) printf("End of file");
if (ferror(fp)) printf("File error");
```

SECTION 14: 12-WEEK STRUCTURED CURRICULUM

Week	Topics	Key Concepts	Practice Activities	Milestone
1	Setup + Problem Solving	Install GCC/Turbo C, algorithms, flowcharts, pseudocode	Draw 5 flowcharts: sum, factorial, Fibonacci, largest, palindrome	Can write algorithm & flowchart for any basic problem
2	Introduction to C + Structure	History, basic structure, #include, main(), comments, compilation	Write 5 programs: Hello World, area of circle, simple calculator, temperature converter, age calculator	Can compile and run a C program independently
3	C Fundamentals I	Data types, variables, constants, #define, typedef, escape sequences	Programs: sizeof all types, constant demonstration, type conversion	Know all basic data types and their ranges by memory
4	C Fundamentals II + Operators	All operators, precedence, pre/post increment, bitwise ops	10 operator problems, trace program outputs, bitwise exercises	Can evaluate any expression with correct precedence
5	Input & Output	printf/scanf with all format specifiers, getchar/putchar, gets/puts	Formatted output programs, alignment, string I/O	Can handle any I/O scenario with correct format specifiers
6	Control Statements I	if, if-else, else-if ladder, nested if, switch-case	Grade calculator, day of week, season finder, leap year	Can solve any decision-based problem
7	Control Statements II (Loops)	for, while, do-while, break, continue, nested loops	Patterns (*, numbers), series sums, prime numbers, palindrome check	Can write all loop types and nested patterns
8	Functions	Prototypes, definitions, call by value/reference, scope, static	Calculator using functions, temperature converter, area functions	Can break any problem into functions
9	Recursion	Recursive thinking, factorial, Fibonacci, power, GCD, tower of Hanoi	5 recursive programs + trace execution	Understand base case and recursive case for any problem
10	Arrays & Strings	1D/2D arrays, string functions, sorting, searching, matrix operations	Bubble sort, linear/binary search, matrix multiplication, string programs	Can implement sorting algorithms and string manipulation
11	Pointers + DMA	Address/dereference, pointer arithmetic, malloc/calloc/free, NULL check	Pointer-based swap, dynamic array, linked structure basics	Understand pointers well enough to debug pointer errors

Week	Topics	Key Concepts	Practice Activities	Milestone
12	Structures + Files + Revision	struct, union, typedef, file modes, fopen/fclose, fprintf/fscanf, binary files	Student record system with file storage, past paper practice	Complete a mini project and attempt one full past paper

SECTION 15: 20+ MINI-PROJECT IDEAS

Projects are the BEST way to learn. Each project below maps to specific syllabus topics and past exam question patterns from Purbanchal University and Pokhara University.

#	Project Name	Concepts Used	Difficulty	Exam Relevance
1	Simple Calculator	Functions, switch-case, loops	■ Beginner	Very High
2	Number Guessing Game	Loops, rand(), conditionals	■ Beginner	Medium
3	Temperature Converter	Functions, float, I/O	■ Beginner	High
4	Grade Calculator & GPA	Arrays, if-else, average	■ Beginner	Very High
5	Currency Converter (NPR ↔ USD ↔ INR)	Functions, float, constants	■ Beginner	Very High
6	Fibonacci & Series Generator	Loops, recursion, arrays	■■ Easy	Very High
7	Star Pattern Printer	Nested loops, formatted output	■■ Easy	High
8	Prime Number Sieve (100 to 200)	Nested loops, arrays, conditions	■■ Easy	Very High
9	String Utilities Library	All string functions, arrays of chars	■■ Easy	High
10	Matrix Calculator (Add/Multiply)	2D arrays, functions, nested loops	■■■ Medium	Very High
11	Bubble Sort Visualizer	Arrays, sorting algorithms, pointers	■■■ Medium	Very High
12	Student Record System (Console)	Structures, arrays of structs, functions	■■■ Medium	Very High
13	Word Counter / Text Analyzer	File I/O, strings, pointers, loops	■■■ Medium	High
14	Stack & Queue (Array-based)	Arrays, functions, DMA	■■■ Medium	Medium
15	Kinetic/Potential Energy Calc ($KP = \frac{1}{2}mv^2 + mgh$)	Math functions, float, I/O	■■ Easy	Very High
16	Binary Search Implementation	Arrays, recursion, pointers	■■■ Medium	High
17	Student DB with File Storage	Structs + Files + DMA + fread/fwrite	■■■■ Hard	Very High
18	Dynamic Memory Int Array	malloc/calloc/realloc/free	■■■ Medium	High
19	Pointer-based Merge Sort	Pointers, recursion, DMA	■■■■ Hard	Medium

#	Project Name	Concepts Used	Difficulty	Exam Relevance
20	Mini Bank System	Structs, files, functions, switch	■■■■ Hard	Very High
21	Library Book Management	Structs, file I/O, search, sort	■■■■ Hard	High
22	Palindrome & Anagram Checker	Strings, arrays, functions	■■ Easy	High
23	Calendar Printer	Loops, conditions, Zeller's formula	■■■ Medium	Medium
24	Tic-Tac-Toe Game	2D arrays, functions, game logic	■■■■ Hard	Medium

SECTION 16: EXAM PREPARATION — PAST PAPER ANALYSIS

16.1 Most Frequently Asked Topics (Purbanchal University 2003–2016)

Topic	Frequency	Mark Weight	Priority
Arrays (1D, 2D, sorting)	Every year	8–10 marks	■ Critical
Pointers & DMA	Every year	8–10 marks	■ Critical
Functions (user-defined, recursion)	Every year	7–10 marks	■ Critical
Structures & Unions	Every year	8–12 marks	■ Critical
File Handling	Every year	7–10 marks	■ Critical
String handling programs	Most years	5–7 marks	■ High
Sorting algorithms	Most years	5–8 marks	■ High
Flowchart & Algorithm	Most years	4–5 marks	■ High
Control statements (loops/conditions)	Most years	4–7 marks	■ High
Output tracing programs	Many years	3–4 marks	■ Medium
Bitwise operators	Some years	3–5 marks	■ Medium
Graphics programming	Some years	3–5 marks	■ Lower

16.2 Typical Exam Question Patterns (BCE 1001 Format)

GROUP-A (4×2=8 marks): Short answer, definition, output tracing

GROUP-B (7×4=28 marks): Programs, explanations, comparisons (choose 7 of ~9)

GROUP-C (3×8=24 marks): Long programs, complex concepts (choose 3 of ~4)

16.3 Important Programs to Master

- 📄 Write a program to sort N numbers in ascending order (bubble sort)
- 📄 Write a program to find factorial using recursion
- 📄 Write a program to check if a string is palindrome
- 📄 Write a program to merge two sorted arrays using pointers
- 📄 Write a program using structure to store and display 10 student records
- 📄 Write a program to read data to a file and display highest scorer
- 📄 Write a program to multiply two matrices
- 📄 Write a program to swap two numbers using pointers (call by reference)

- 🍰 ■ Write a program to count vowels and consonants using pointers
- 🍰 ■ Write a program for Fibonacci series (both iterative and recursive)
- 🍰 ■ Write a program for dynamic memory allocation for n numbers
- 🍰 ■ Write a program to convert US dollars to Nepali rupees using functions

■ **Tip:** Always write the algorithm/flowchart FIRST in exams, even if not asked — it helps you organize your thoughts before coding.

■ **Note:** In exams, partial marks ARE given for correct logic even if syntax is slightly wrong. Show your thinking clearly!